SECURITY ENGINEERING PROJECT REPORT

From Passwords to Passkeys

An interactive guide to authentication, phishing and MFA

Course	COMP6441 Security Engineering
Student	Name: _____ zID: _____
Audience	Security beginners and general university users
Prepared	20 July 2026

Submission integrity note

The artefact, functional tests and evidence index are complete. No human-participant result has been invented. The supplied evaluation protocol is ready to run; any future participant results must be inserted only after informed, anonymous testing.

Executive summary

AuthLab is a privacy-first, browser-based teaching resource that explains why authentication failures are system failures rather than simply examples of careless users. The project translates four connected security concepts—identity and authorisation, password reuse and credential stuffing, phishing and impersonation, and phishing-resistant passkeys—into short interactive activities for beginners. The central engineering claim is that a login method should be judged by its protocol properties and threat model, not only by whether it adds another step.

The final artefact is a responsive static web application with a six-question pre-test and post-test, a harmless credential-stuffing simulation, four phishing decisions, an MFA/passkey threat matrix, a passkey ceremony diagram and an optional live WebAuthn demonstration. It stores progress in the learner's own browser, requests no real password or identifying information, and exports anonymous CSV/JSON evidence for a facilitator. A workbook, five-minute presentation, narrated video and evaluation protocol extend the site into a complete teaching package.

Technical validation covered production build success, rendered-page tests, responsive visual inspection and a full browser path. The developer self-test completed every module and produced a 6/6 pre-test and 6/6 post-test; this is evidence that the correct-answer and export path work, not evidence of learning. No human learner dataset was supplied, so effectiveness with the intended audience remains an explicit open evaluation question.

1. Project aim and research question

The project asks: how can a beginner-facing resource reveal the engineering differences between passwords, generic MFA and passkeys without collecting credentials or teaching unsafe habits? The answer is organised around three observable decisions: what claim is made about identity, what proof an authenticator provides, and what the relying party is permitted to authorise.

> **Project outcome**
>
> A learner can inspect one complete authentication threat chain, explain why copied visual appearance is weak evidence, compare authenticator classes against concrete attacks, and describe how origin-bound public-key credentials resist phishing.

2. Security background: claims, proofs and decisions

2.1 Identity, authentication and authorisation

Identity is the account or subject being referred to; authentication is the process used to establish confidence that a claimant controls an approved authenticator; authorisation is the decision about what that authenticated subject may do. Combining these into one vague idea—'logging in'—obscures where controls fail. A copied sign-in page can collect an authentic password while sending it to the wrong verifier; successful authentication can still lead to excessive authorisation; and a legitimate identity record can be paired with a weak recovery path.

NIST SP 800-63B treats phishing resistance as a property of the authentication protocol. Passwords are not replay resistant, and manually entered one-time passwords are not phishing resistant. A verifier-name-binding mechanism such as WebAuthn can be phishing resistant because the authenticator output is cryptographically bound to the verified relying-party identifier [1]. This distinction guided both the learning sequence and the threat matrix.

2.2 Passwords and credential stuffing

Password reuse turns a breach at one service into an authentication attack against unrelated services. Credential stuffing automates the use of previously stolen username/password pairs across many sites [4]. The key engineering lesson is propagation: the second service may have no breach of its own, yet a reused secret allows the attacker to cross a trust boundary. The simulation therefore uses dummy accounts and example.test domains to show two successful takeovers from one leaked credential without imitating a real brand or asking for a real password.

Current NIST guidance favours long passwords, blocklists of commonly used or compromised values, password-manager support and rate limiting; it rejects arbitrary composition rules and routine password changes without evidence of compromise [2]. AuthLab does not reduce the lesson to 'make the password complicated'. It shows that even a strong human-memorised secret remains replayable and phishable.

Layer	Question	Typical failure
Identity	Who is the account about?	Ambiguous or poorly recovered account
Authentication	What proof is accepted?	Replayable or phishable proof
Authorisation	What action is allowed?	Excess privilege after valid sign-in

3. Security background: phishing, MFA and passkeys

3.1 Why visual similarity is not identity

Phishing succeeds when the learner treats page appearance, urgency or a familiar logo as proof of origin. CISA frames phishing as the opening phase of an attack cycle and recommends layered technical and human controls [5]. AuthLab's four scenarios force the learner to identify the registered domain, compare the requested action with the context, and select a safer recovery route such as opening a known bookmark or trusted application independently.

3.2 MFA is a family, not a guarantee

The label 'MFA' combines mechanisms with different failure modes. SMS codes can be exposed through number reassignment or social engineering; TOTP codes can be relayed in real time; approval pushes can be abused through fatigue or context confusion; and recovery channels can undercut a strong primary factor. The project therefore compares replay resistance, phishing resistance and user-verification behaviour rather than assigning a single security score.

3.3 Passkeys and the WebAuthn ceremony

A passkey is a FIDO credential based on asymmetric cryptography. The relying party stores a public key, while the authenticator protects the private key. During registration the browser mediates credential creation for the current relying party. During authentication the server supplies a fresh challenge, the authenticator verifies the user according to policy, and a signed assertion is returned. The private key is not sent to the website [3].

WebAuthn scopes credentials to a relying party through the RP ID and origin checks. Authenticator data includes an rpIdHash, flags and a signature counter field; the relying party verifies the challenge, origin, RP binding and signature [6]. This verifier-name binding prevents a credential created for one domain from being used by a look-alike domain, assuming the browser, authenticator and platform trust boundaries hold.

Authenticator	Replay resistant	Phishing resistant	Primary concern
Password	No	No	Reuse, disclosure and offline/online guessing
SMS OTP	Limited	No	Relay, telecom and recovery weaknesses
TOTP	Yes (short-lived)	No	Real-time relay to the attacker
Push approval	Usually	Not inherently	Fatigue and poor transaction context
Passkey/WebAuthn	Yes	Yes	Device, recovery and ecosystem governance

4. Instructional design and method

The selected project category was Teach Something. The intended audience is a beginner who uses online accounts but does not need prior cryptography or programming knowledge. The instructional design follows a concrete-to-abstract sequence: first separate the three login decisions; then observe credential reuse across sites; then inspect how a phishing site changes the verifier; finally compare mechanisms and reveal how passkeys alter the protocol.

4.1 Learning objectives

- Distinguish identity, authentication and authorisation in a realistic sign-in flow.
- Explain how one leaked, reused password can enable credential stuffing elsewhere.
- Identify the registered domain and select a safer action in phishing scenarios.
- Compare passwords, OTP, push MFA and passkeys against replay and phishing threats.
- Describe registration and authentication in WebAuthn without claiming that a private key leaves the authenticator.

4.2 Evidence strategy

The website contains matched six-question pre- and post-tests. Questions cover concepts rather than recall of interface labels. Module interactions provide completion evidence, and anonymous exports contain scores, scenario decisions and completion state. This supports a small pre/post study while preserving a clean boundary: the application records outcomes selected by the learner but does not infer personal identity, security skill or demographic attributes.

4.3 Evaluation protocol

1. Recruit 3–5 consenting learners from the stated beginner audience; do not collect names, emails or real credentials.
2. Ask each learner to complete the pre-test, four modules and post-test independently while the facilitator records only non-identifying observations.
3. Export one anonymous results file per participant and assign a random participant code outside the site.

4. Report individual and aggregate score changes, recurring misconceptions and usability feedback without claiming causation from a small convenience sample.
5. Delete raw exports after marking unless retention is explicitly agreed and ethically justified.

Evidence boundary

The packaged self-test proves that the flow, scoring and export work. It must not be presented as learner-effectiveness evidence. Human results belong in the report only after the protocol is run.

5. Artefact architecture

AuthLab is implemented as a client-rendered Next.js/React application with no server-side account system. Four instructional modules, a quiz/evidence layer and a download/video section share one content model, while optional WebAuthn calls remain scoped to the deployed origin and can be skipped on unsupported platforms.

Component	Security responsibility	Evidence produced
Quiz engine	Scores six concept questions before and after learning	Local score and anonymous export
Password simulation	Uses only dummy accounts and example.test domains	Safe propagation demonstration
Phishing scenarios	Explains domain and action evidence	Four recorded decisions
Threat matrix	Separates replay and phishing resistance	Comparable mechanism model
WebAuthn demo	Invokes browser credential APIs for the current RP	Public metadata/status only
Storage/export	Keeps progress client-side; no analytics endpoint	CSV/JSON chosen by learner

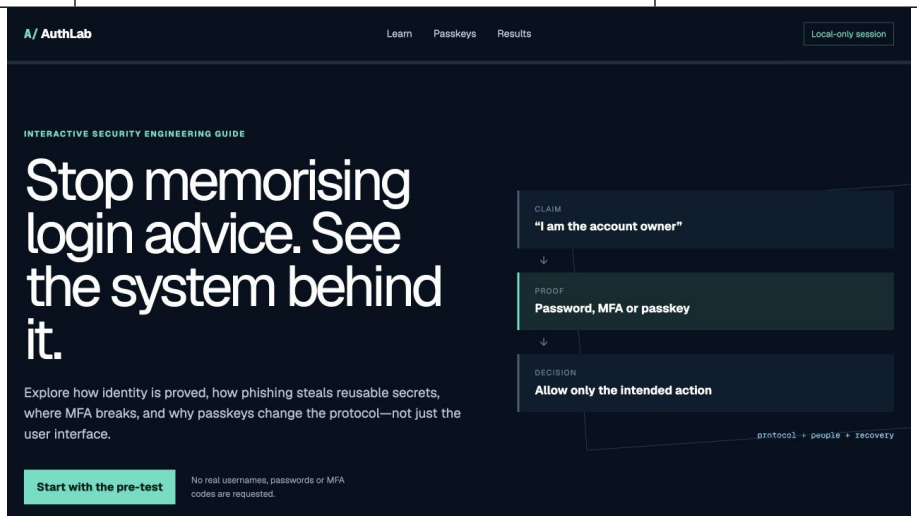

Figure 1. AuthLab begins with a privacy notice, pre-test and four-module learning path.

6. Module evidence: password reuse and credential stuffing

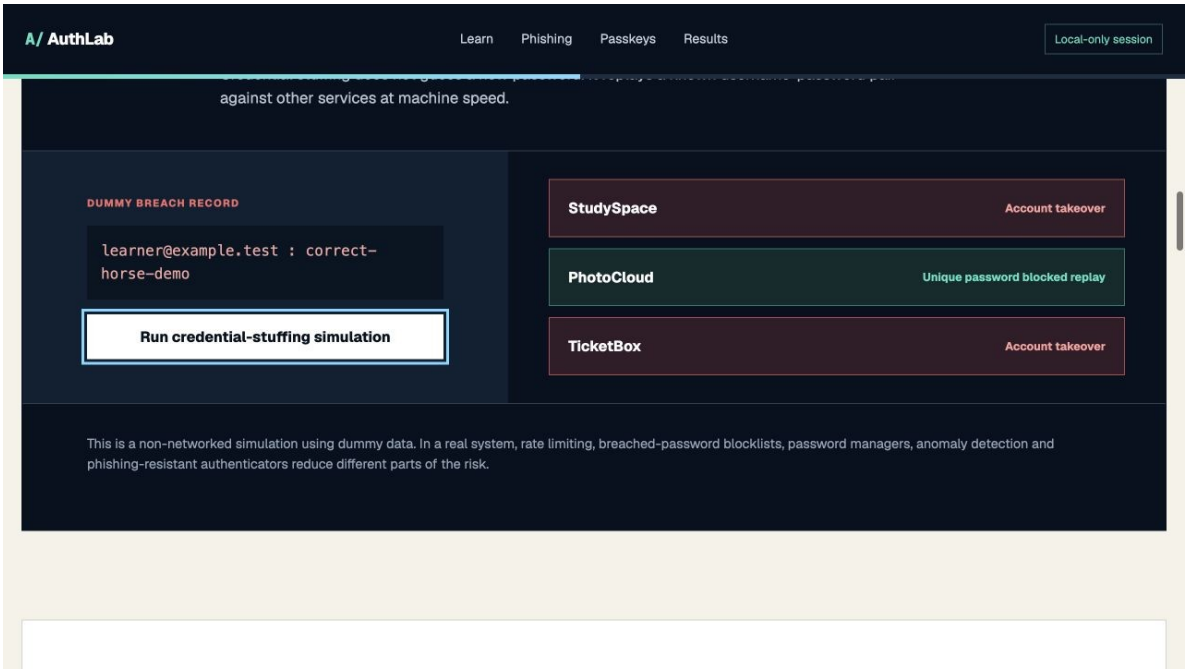

Figure 2. Credential-stuffing simulation with fictitious accounts and domains.

The simulation begins with one fictitious credential leak and asks the learner to run a harmless check against other dummy services. The resulting account takeovers expose the dependency created by secret reuse. The sequence is intentionally causal: claim, compromised proof, automated reuse and cross-service impact. It avoids real websites, copied brands and operational instructions that could be repurposed for attack.

Engineering analysis

The important variable is not password complexity in isolation but whether the same verifier accepts the same reusable secret. Rate limiting, breached-password blocklists, risk-based detection and unique passwords reduce parts of the threat, while passkeys remove the shared/replayable secret from the server-verifier relationship. Recovery remains a separate system that must be threat-modelled.

> **Beginner checkpoint**
>
> If a password from Service A also opens Service B, Service B can be compromised without its own password database being breached. The attacker is replaying proof, not breaking Service B's cryptography.

7. Module evidence: phishing and impersonation

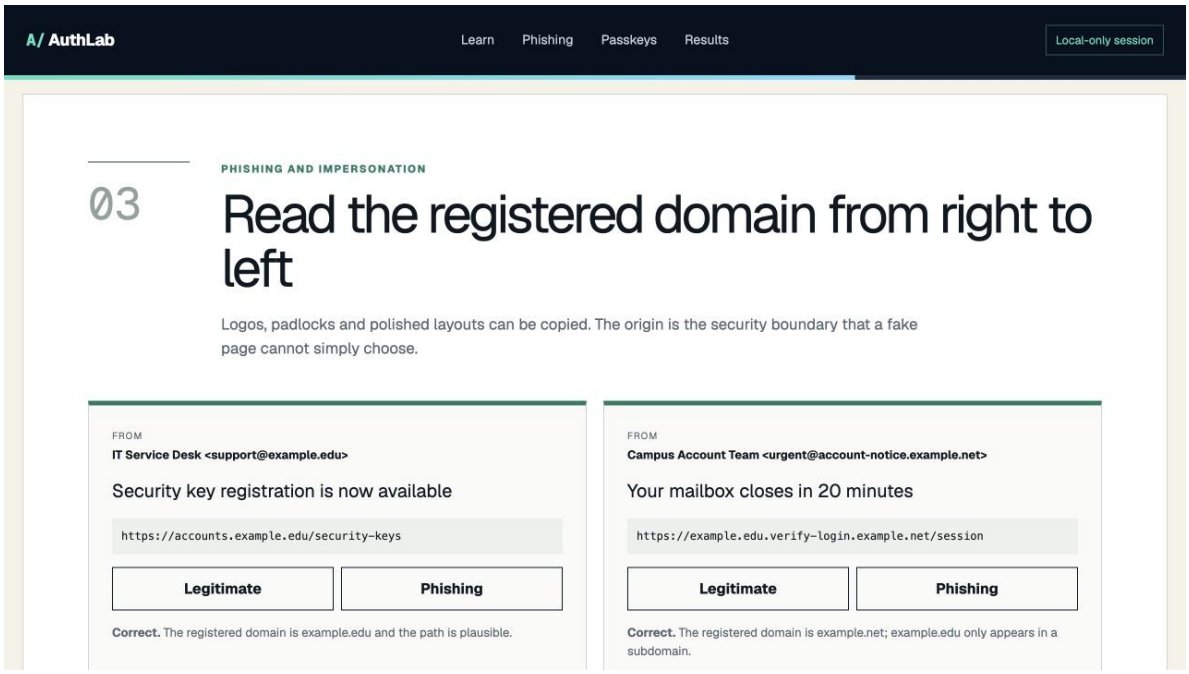

Figure 3. Scenario answer with the decisive domain and safer action explained.

Each scenario pairs a plausible message with a concrete decision. Feedback identifies the registered domain or mismatched action and explains how to reach the service through a trusted path. The activity does not teach a fragile checklist such as ‘look for spelling mistakes’. Modern phishing may be grammatically polished and visually exact, so the engineering focus is verifier identity, origin and requested transaction.

Analysis of limitations

Domain inspection is necessary but not sufficient. Internationalised names, compromised legitimate accounts, malicious subdomains and OAuth consent abuse can complicate decisions. Users may also encounter attacks inside trusted collaboration tools rather than email. The module therefore treats URL evidence as one control in a layered response: verify context, avoid embedded sign-in paths, use a trusted application/bookmark, and report suspicious activity.

> **Design principle**
>
> The page can copy pixels. It cannot make a WebAuthn credential scoped to the legitimate relying party sign an assertion for the attacker’s origin.

8. Passkeys: phishing resistance by design

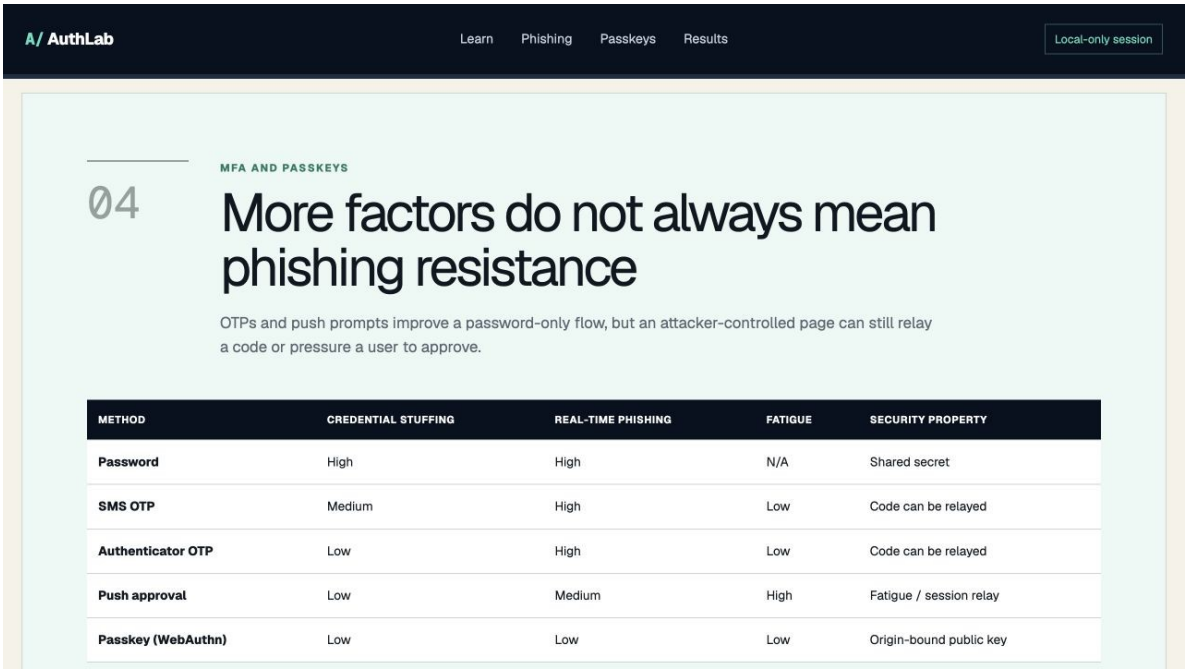

Figure 4. Threat matrix and passkey ceremony used in the final module.

The threat matrix prevents an imprecise conclusion that 'MFA fixes phishing'. It asks whether an authenticator output can be replayed, whether the protocol binds that output to the intended verifier, and what recovery or device risks remain. The passkey flow then visualises server challenge generation, browser/RP validation, local user verification and signed assertion verification.

Optional WebAuthn demonstration

The live demonstration calls navigator.credentials.create() to request a discoverable public-key credential for the current relying party and navigator.credentials.get() to request an assertion. The interface displays status and non-secret metadata only. It does not display or transmit the private key. A deployed HTTPS origin is preferable because WebAuthn is restricted to secure contexts, although browsers normally treat localhost as a development exception.

Passkeys do not eliminate all account risk. Device compromise, weak local user verification, insecure account recovery, malicious relying-party logic and ecosystem governance remain relevant. The stronger claim is narrower and testable: a correctly implemented origin-bound WebAuthn ceremony resists credential phishing better than passwords or manually entered OTPs.

9. Verification and current results

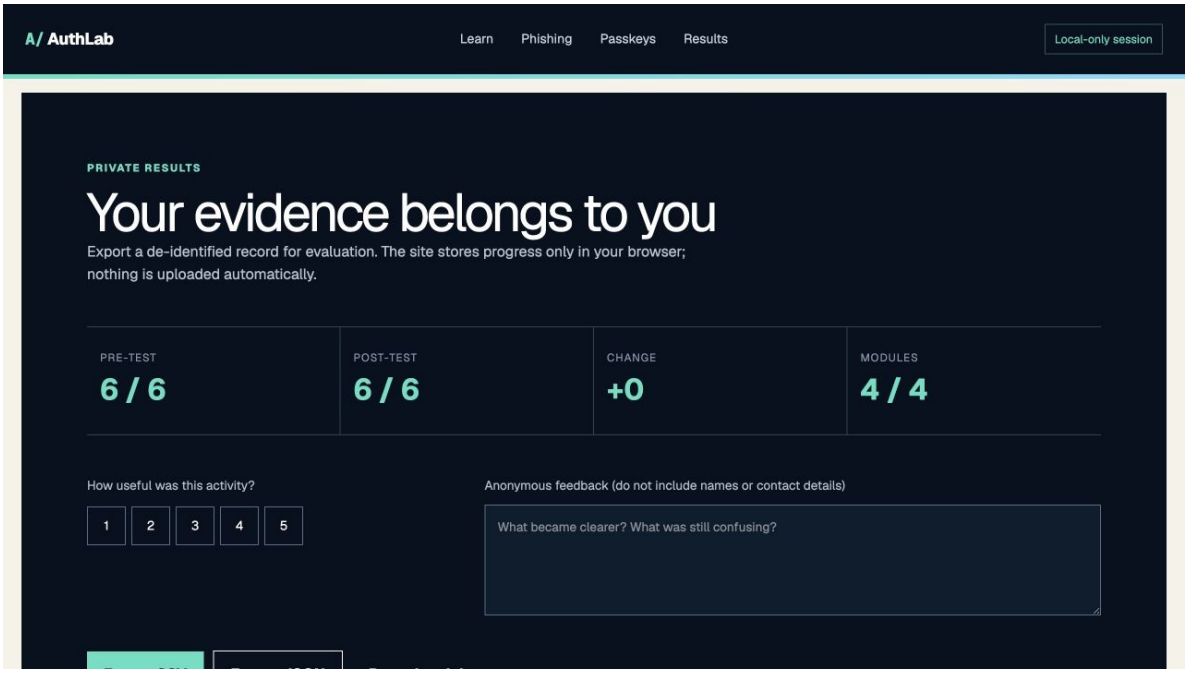

Figure 5. Completed internal path test. Scores show a functional perfect-answer route, not a measured learning gain.

Verification	Result	Interpretation
Production build	Pass	Application compiles for production
Rendered HTML tests	Pass	Expected title, structure and content are present
Browser path	Pass	All four modules, both quizzes and export state complete
Responsive visual QA	Pass	Hero, modules, tables and results inspected
Video probe	Pass	H.264 1920×1080, AAC audio, English subtitle track
Human learning study	Not yet run	Effectiveness remains unverified

The internal correct-answer path produced 6/6 before and after. A constant score is expected because the tester deliberately selected correct answers to exercise the system. It is not meaningful to calculate learning gain from this run. The results screen and export are ready for authentic participant use, where both improvement and persistent misconceptions can be analysed.

Success measures for the human test

- Score change per participant and item-level error change, reported without hiding negative or zero change.
- Ability to explain why TOTP may be replay resistant yet phishable through real-time relay.
- Ability to distinguish the page's visual appearance from the relying-party origin.
- Ability to describe what the server stores for a passkey and what remains with the authenticator.

10. Security, privacy and ethics

A teaching artefact about authentication can create harm if it solicits real credentials, impersonates real services, stores identifiable behaviour or presents offensive security steps without safeguards. AuthLab addresses these risks through data minimisation, fictitious examples, local processing and explicit learner warnings.

Risk	Control	Residual limitation
Learner submits a real password	No password input; all credentials are fixed dummy values	Learner may still disclose information outside the tool
Copied brand enables impersonation	Fictitious names and example.test domains	Scenarios cannot represent every attack channel
Participant identification	No names, emails, analytics or server database	Exported files may become identifying if combined externally
Passkey misconception	Shows public/private-key boundary and RP binding	Platform UI and synchronisation vary by ecosystem
Inflated evaluation claim	Self-test labelled as functional evidence only	Human effectiveness remains to be measured
AI-generated error	Primary-source research, transparent acknowledgement and required student review	Final responsibility remains with the submitter

Ethical participant procedure

A facilitator should explain that participation is voluntary, that the exercise is educational rather than an assessment of personal competence, and that no real account data should be entered. Participants should be able to stop without penalty. Results should be summarised proportionately; a small convenience sample can reveal usability problems but cannot support broad claims about population-level effectiveness.

Safe-use rule

Never ask a participant to demonstrate a personal password, recovery method, live MFA code or existing passkey. Use only the fictional examples supplied with AuthLab.

11. Challenges, progression and reflective analysis

11.1 From advice to an engineering model

The proposal began as a broad guide to authentication, phishing and MFA. The central improvement was to organise the material around a protocol model—claim, proof and decision—rather than a collection of tips. That change made the password, phishing and passkey modules comparable and allowed the threat matrix to evaluate properties instead of product labels.

11.2 Separating MFA deployment from phishing resistance

A second challenge was avoiding the common but technically weak statement that any MFA prevents phishing. Reviewing NIST's explicit treatment of phishing resistance forced a narrower distinction between short-lived output, replay resistance and verifier-name binding. The final site therefore presents SMS, TOTP, push and passkeys as different mechanisms with different residual risks.

11.3 Demonstrating passkeys without collecting secrets

A live WebAuthn feature is more impressive than a static diagram, but it creates device, browser and secure-context dependencies. The design resolves this tension by making the real browser ceremony optional, showing only non-secret metadata, and providing a review-completion control so that learners on unsupported platforms are not blocked.

11.4 Evidence honesty

The requirement to test the teaching resource creates pressure to show a learning improvement. The responsible response is to separate automated and developer testing from participant research. The artefact can be complete while the human-evaluation claim remains open. This is both a project limitation and a security-engineering lesson: evidence must be scoped to what the test actually establishes.

Growth demonstrated by the artefact

- Threat modelling moved from general user behaviour to replay, relay, origin binding, recovery and authorisation boundaries.
- Security communication moved from warning lists to observable mechanisms and learner decisions.
- Privacy moved from a policy statement to an architectural constraint: local state, no accounts and anonymous export.
- Evaluation moved from a nominal pre/post idea to a reproducible protocol with an explicit non-participant self-test label.

12. Strengths, weaknesses and conclusion

Strengths

AuthLab connects conceptual depth with a demonstrable artefact. It exposes the protocol mechanics behind passkeys, uses safe simulations, supports an authentic teaching session, minimises data and packages evidence across a website, workbook, report, presentation and video. The repeated claim/proof/decision structure makes the four modules coherent rather than merely adjacent.

Weaknesses and future work

The present version has no verified human learning dataset, screen-reader user study or cross-browser WebAuthn compatibility matrix. Its static architecture cannot demonstrate server-side challenge persistence, credential database design or production recovery. Future work should add a controlled reference relying party, accessibility testing with users, internationalised-domain examples, recovery-path threat modelling and an ethically approved participant evaluation.

Conclusion

Passwords, MFA and passkeys should not be compared by counting user steps. They should be compared by the evidence they produce, the verifier to which that evidence is bound, the attacks they resist and the recovery systems that surround them. AuthLab makes those properties visible to a beginner without requesting real secrets. The result is a technically grounded teaching resource and a documented evaluation plan whose remaining uncertainty—human effectiveness—is stated rather than concealed.

References

[1] NIST, SP 800-63B: Authentication and Authenticator Management (final). Official source (accessed 20 July 2026).

[2] NIST, Passwords, SP 800-63B implementation resources. Official source (accessed 20 July 2026).

[3] FIDO Alliance, Passkeys. Official source (accessed 20 July 2026).

[4] OWASP, Credential Stuffing. Official source (accessed 20 July 2026).

[5] CISA, Phishing Guidance: Stopping the Attack Cycle at Phase One, 2025. Official source (accessed 20 July 2026).

[6] W3C, Web Authentication: An API for accessing Public Key Credentials—Level 3. Official source (accessed 20 July 2026).

[7] FIDO Alliance, Passkeys: The Journey to Prevent Phishing Attacks. Official source (accessed 20 July 2026).

Appendix A. Evidence and submission record

Evidence item	Location / status	What it establishes
Interactive guide	authlab-site/	Complete four-module teaching artefact
Build and HTML tests	authlab-site build/test output	Production compilation and required page structure
Visual evidence	evidence/screenshots/00–07	Rendered interface and completed path
Narrated video	deliverables/video/AuthLab_Demo.mp4	90-second walkthrough with audio/subtitles
Workbook	deliverables/workbook/	Facilitator and learner activities, answer key and protocol
Presentation	deliverables/presentation/	Five-minute assessment summary and speaker notes
Human participant evidence	Not yet collected	Must be added only after the supplied protocol is run

Generative AI acknowledgement

OpenAI Codex was used to assist with primary-source research synthesis, application code, document layout, slide and video production, and quality assurance. All security claims and references should be checked by the submitting student, who remains responsible for understanding, verifying and adapting the work. Generative AI was not used to fabricate human-participant data.

Required student verification before submission

- Enter the correct name and zID on the cover and rename the final report to Report-[your_zID].pdf.
- Read the entire report and confirm that the reflective account matches the work you can personally explain.
- Run the participant protocol if the assessment requires evidence of teaching effectiveness; insert only authentic de-identified results.
- Verify the deployed website/video links and retain a stable GitHub or OneDrive copy if required by the course submission rules.
- Keep this AI acknowledgement or replace it with the course-approved disclosure format.